ZAINA RAHEEN A.P (241801328)

Ex5a: Information Extraction using NLTK

2. Install and Import Required Libraries

```
import pandas as pd
import nltk

print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

3. Import NLTK Resources

```
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('maxent_ne_chunker')
nltk.download('words')

print("NLTK resources downloaded successfully.")
```

```
NLTK resources downloaded successfully.
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package maxent_ne_chunker to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package maxent_ne_chunker is already up-to-date!
[nltk_data] Downloading package words to /root/nltk_data...
[nltk_data]   Package words is already up-to-date!
```

4. Load the Dataset

```
try:
    df = pd.read_csv('/content/sample_data/Reviews.csv', engine='python', on_bad_lines='skip') # Added on_bad_lines='skip'
except FileNotFoundError:
    try:
        df = pd.read_csv('/tmp/Reviews.csv', engine='python', on_bad_lines='skip') # Added on_bad_lines='skip'
    except FileNotFoundError:
        print("Error: 'Reviews.csv' not found. Please upload the file or specify the correct path.")
        df = pd.DataFrame()
if not df.empty:
    review_column = 'Text'
    if review_column in df.columns:
        reviews = df[[review_column]].copy()
        reviews.dropna(inplace=True)
        reviews = reviews.head(10000)
        print(f"Dataset loaded with {len(reviews)} reviews after cleaning and limiting.")
        print("First 5 reviews:")
        print(reviews.head())
    else:
        print(f"Error: Column '{review_column}' not found in the dataset. Available columns: {df.columns.tolist()}")
        reviews = pd.DataFrame()
```

```
else:
    print("DataFrame is empty, unable to proceed with review processing.")
```

```
Dataset loaded with 10000 reviews after cleaning and limiting.
First 5 reviews:
                                                Text
0  I have bought several of the Vitality canned d...
1  Product arrived labeled as Jumbo Salted Peanut...
2  This is a confection that has been around a fe...
3  If you are looking for the secret ingredient i...
4  Great taffy at a great price.  There was a wid...
```

5. Do the preprocessing step:

```
import string

if not reviews.empty:
    reviews['Text_processed'] = reviews['Text'].str.lower()
    translator = str.maketrans('', '', string.punctuation)
    reviews['Text_processed'] = reviews['Text_processed'].apply(lambda x: x.translate(translator))
    print("Preprocessing (lowercasing and punctuation removal) complete.")
    print("First 5 processed reviews:")
    print(reviews[['Text', 'Text_processed']].head())
else:
    print("Reviews DataFrame is empty, skipping preprocessing.")
```

```
Preprocessing (lowercasing and punctuation removal) complete.
First 5 processed reviews:
                                                Text  \
0  I have bought several of the Vitality canned d...
1  Product arrived labeled as Jumbo Salted Peanut...
2  This is a confection that has been around a fe...
3  If you are looking for the secret ingredient i...
4  Great taffy at a great price.  There was a wid...

                                      Text_processed
0  i have bought several of the vitality canned d...
1  product arrived labeled as jumbo salted peanut...
2  this is a confection that has been around a fe...
3  if you are looking for the secret ingredient i...
4  great taffy at a great price  there was a wide...
```

6. Perform tokenization and remove stopwords, non-alphabetic tokens.

```
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

if not reviews.empty:
    stop_words = set(stopwords.words('english'))

    def tokenize_and_filter(text):
        tokens = word_tokenize(text)
        filtered_tokens = [word for word in tokens if word.isalpha() and word not in stop_words]
        return filtered_tokens
    reviews['Text_tokens'] = reviews['Text_processed'].apply(tokenize_and_filter)
    print("Tokenization, stopword removal, and non-alphabetic token removal complete.")
    print("First 5 processed and tokenized reviews (first 10 tokens each):")
    for i, row in reviews.head().iterrows():
        print(f"Original: {row['Text_processed']}")
        print(f"Tokens: {row['Text_tokens'][:10]}...")
else:
    print("Reviews DataFrame is empty, skipping tokenization and filtering.")
```

```
Tokenization, stopword removal, and non-alphabetic token removal complete.
First 5 processed and tokenized reviews (first 10 tokens each):
Original: i have bought several of the vitality canned dog food products and have found them all to be of good quality the produ
Tokens: ['bought', 'several', 'vitality', 'canned', 'dog', 'food', 'products', 'found', 'good', 'quality']...
Original: product arrived labeled as jumbo salted peanutsthe peanuts were actually small sized unsalted not sure if this was an
Tokens: ['product', 'arrived', 'labeled', 'jumbo', 'salted', 'peanutsthe', 'peanuts', 'actually', 'small', 'sized']...
Original: this is a confection that has been around a few centuries  it is a light pillowy citrus gelatin with nuts  in this cas
Tokens: ['confection', 'around', 'centuries', 'light', 'pillowy', 'citrus', 'gelatin', 'nuts', 'case', 'filberts']...
Original: if you are looking for the secret ingredient in robitussin i believe i have found it  i got this in addition to the ro
Tokens: ['looking', 'secret', 'ingredient', 'robitussin', 'believe', 'found', 'got', 'addition', 'root', 'beer']...
Original: great taffy at a great price  there was a wide assortment of yummy taffy  delivery was very quick  if your a taffy lov
Tokens: ['great', 'taffy', 'great', 'price', 'wide', 'assortment', 'yummy', 'taffy', 'delivery', 'quick']...
```

7. Perform Parts of Speech (POS) tagging.

```
if not reviews.empty:
    reviews['Text_pos_tagged'] = reviews['Text_tokens'].apply(nltk.pos_tag)
    print("POS tagging complete.")
    print("First 5 tokenized and POS tagged reviews (first 10 tags each):")
    for i, row in reviews.head().iterrows():
        print(f"Tokens: {row['Text_tokens'][:10]}...")
        print(f"POS Tags: {row['Text_pos_tagged'][:10]}...")
else:
    print("Reviews DataFrame is empty, skipping POS tagging.")
```

```
POS tagging complete.
First 5 tokenized and POS tagged reviews (first 10 tags each):
Tokens: ['bought', 'several', 'vitality', 'canned', 'dog', 'food', 'products', 'found', 'good', 'quality']...
POS Tags: [('bought', 'VBD'), ('several', 'JJ'), ('vitality', 'NN'), ('canned', 'VBD'), ('dog', 'JJ'), ('food', 'NN'), ('product
Tokens: ['product', 'arrived', 'labeled', 'jumbo', 'salted', 'peanutsthe', 'peanuts', 'actually', 'small', 'sized']...
POS Tags: [('product', 'NN'), ('arrived', 'VBD'), ('labeled', 'JJ'), ('jumbo', 'NN'), ('salted', 'VBD'), ('peanutsthe', 'NN'), (
Tokens: ['confection', 'around', 'centuries', 'light', 'pillowy', 'citrus', 'gelatin', 'nuts', 'case', 'filberts']...
POS Tags: [('confection', 'NN'), ('around', 'IN'), ('centuries', 'NNS'), ('light', 'JJ'), ('pillowy', 'NNS'), ('citrus', 'VBP'),
Tokens: ['looking', 'secret', 'ingredient', 'robitussin', 'believe', 'found', 'got', 'addition', 'root', 'beer']...
POS Tags: [('looking', 'VBG'), ('secret', 'JJ'), ('ingredient', 'NN'), ('robitussin', 'NN'), ('believe', 'VBP'), ('found', 'VBN'
Tokens: ['great', 'taffy', 'great', 'price', 'wide', 'assortment', 'yummy', 'taffy', 'delivery', 'quick']...
POS Tags: [('great', 'JJ'), ('taffy', 'JJ'), ('great', 'JJ'), ('price', 'NN'), ('wide', 'JJ'), ('assortment', 'NN'), ('yummy', '
```